

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	1
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	3
ВВЕДЕНИЕ	4
1 АНАЛИТИЧЕСКАЯ ЧАСТЬ	6
1.1 Обзор предметной области	6
1.1.1 Подходы к защите контента	8
1.2 Анализ аналогов	9
1.3 Постановка задачи и требования	9
1.3.1 Функциональные требования	9
1.3.2 Нефункциональные требования	10
1.3.3 Действующие лица и сценарии использования	10
2 ПРОЕКТНАЯ ЧАСТЬ	12
2.1 Общая архитектура	12
2.2 Модель данных	13
2.3 Проектирование защиты контента	15
2.4 Проектирование транскодинга	16
2.5 Проектирование API	18
2.6 Развёртывание	19
3 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ	20
3.1 Go для стриминга и транскодинга	20
3.2 Django и DRF для бэкенда	20
3.3 Nginx	21
3.4 Вспомогательные технологии	21
3.5 Соответствие техническому заданию	22
4 РЕАЛИЗАЦИЯ	23
4.1 Модель пользователя и права доступа	23
4.2 Выпуск и проверка токена доступа	24
4.3 Защищённая раздача HLS	25
4.4 Транскодинг в адаптивный HLS	25
4.5 Nginx: проксирование и кэш-CDN	26
4.6 Фронтенд	27
4.7 Контейнеризация и развёртывание	27
4.8 Подсчёт результата теста	29
4.9 Учёт прогресса	29
5 ТЕСТИРОВАНИЕ	30
5.1 Модульное тестирование	30
5.2 Интеграционная проверка	32
5.3 Безопасность	33
5.4 Анализ производительности стриминга	34
ЗАКЛЮЧЕНИЕ	36
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	37

ПРИЛОЖЕНИЕ А. Структура репозитория и история разработки	38
ПРИЛОЖЕНИЕ Б. Документация API	38
ПРИЛОЖЕНИЕ В. Экранные формы	40
ПРИЛОЖЕНИЕ Г. Руководство по запуску и проверке	42

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

Сокращение	Расшифровка
HLS	HTTP Live Streaming — протокол адаптивного потокового вещания
ABR	Adaptive Bitrate — адаптивный битрейт
VOD	Video on Demand — видео по запросу
CDN	Content Delivery Network — сеть доставки контента
DRM	Digital Rights Management — управление цифровыми правами
API	Application Programming Interface — программный интерфейс
REST	Representational State Transfer — архитектурный стиль API
JWT	JSON Web Token — токен аутентификации
HMAC	Hash-based Message Authentication Code — код аутентификации сообщений
DRF	Django REST Framework
ORM	Object-Relational Mapping — объектно-реляционное отображение
SAST	Static Application Security Testing — статический анализ безопасности
SCA	Software Composition Analysis — анализ состава ПО (зависимостей)
CI	Continuous Integration — непрерывная интеграция
CSRF	Cross-Site Request Forgery — межсайтовая подделка запроса

ВВЕДЕНИЕ

Дистанционное обучение и видео-контент стали неотъемлемой частью современного образования. Рост числа онлайн-платформ предъявляет высокие требования к доставке видео: оно должно воспроизводиться без задержек на устройствах с разной пропускной способностью канала и при этом быть защищено от несанкционированного скачивания. Эти задачи решаются технологией адаптивного потокового вещания HLS (HTTP Live Streaming) в связке с механизмами контроля доступа к контенту.

Настоящая курсовая работа выполнена по варианту 33 «Платформа для онлайн-курсов с видео-стримингом». Согласно заданию, система реализуется на стеке Go + Python + Nginx: Go-сервис обеспечивает стриминг видео с адаптивным битрейтом (HLS), Python-бэкенд (Django) управляет курсами, пользователями и тестами, предусмотрена интеграция с CDN и защита контента.

Цель работы — продемонстрировать способность создавать сложный программный продукт с использованием современных языков программирования, технологий и архитектурных подходов, пройдя полный цикл разработки: от идеи до развёртывания.

Для достижения цели поставлены следующие задачи:

1. провести анализ предметной области и сформулировать требования к программному продукту;
2. спроектировать архитектуру приложения с использованием современных методологий;
3. реализовать программный продукт с применением изученных технологий;
4. обеспечить качество кода через тестирование и статический анализ;
5. подготовить документацию и провести презентацию результатов.

Объект исследования — процесс доставки и защиты учебного видео-контента. **Предмет** — методы адаптивного стриминга и контроля доступа в распределённой системе из микросервисов.

Результатом работы является работающий программный продукт, разворачиваемый одной командой через Docker Compose, с покрытием модульными тестами, статическим анализом безопасности и комплектом документации. При разработке нейтрально применялись AI-ассистенты для генерации шаблонного кода, тестов и документации, что соответствует требованию методических указаний об интеграции ИИ-инструментов.

Пояснительная записка состоит из пяти разделов. В аналитической части исследуется предметная область и формулируются требования. В проектной части разрабатывается архитектура и приводятся диаграммы. В технологической части обосновывается выбор инструментов. В разделе реализации описываются ключевые компоненты с фрагментами кода. В разделе тестирования приводятся результаты проверок, анализ безопасности и производительности. Завершают записку заключение, список литературы и приложения с документацией API и экранными формами.

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 Обзор предметной области

Платформа онлайн-курсов — это веб-система, предоставляющая учебные материалы (видео-уроки, тексты, тесты) и управляющая доступом пользователей к ним. Ключевым техническим элементом такой платформы является подсистема доставки видео.

Передача видео по сети возможна двумя способами: прямое скачивание файла (progressive download) и адаптивное потоковое вещание. Прямое скачивание не позволяет управлять качеством под канал пользователя и плохо защищает контент — файл доступен по постоянной ссылке. Поэтому индустриальным стандартом стал HLS.

HLS (HTTP Live Streaming) — протокол потоковой передачи поверх обычного HTTP, разработанный Apple (RFC 8216). Исходное видео нарезается на короткие сегменты (обычно 2–10 секунд) в формате MPEG-TS (файлы .ts), а перечень сегментов описывается текстовым плейлистом .m3u8. Для адаптивного битрейта готовится несколько вариантов качества (например, 360p, 720p, 1080p), а главный плейлист (master playlist) перечисляет их с указанием битрейта. Плеер сам выбирает подходящее качество и переключается между вариантами в зависимости от текущей скорости соединения.

Структурно главный плейлист (master playlist) перечисляет доступные варианты качества, а плейлисты вариантов (media playlist) — конкретные сегменты. Пример главного плейлиста с тремя уровнями качества:

Листинг 1 — пример главного плейлиста HLS (master.m3u8)

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-STREAM-INF:BANDWIDTH=928000,RESOLUTION=640x360
360p/index.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3128000,RESOLUTION=1280x720
720p/index.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=5600000,RESOLUTION=1920x1080
```

Выбор длительности сегмента — компромисс: короткие сегменты (2–4 с) уменьшают задержку старта и ускоряют переключение качества, но увеличивают число HTTP-запросов и накладные расходы; длинные сегменты эффективнее по сети, но менее отзывчивы. В работе принята длительность 4 секунды как сбалансированное значение для VOD (video on demand).

Адаптивный битрейт (Adaptive Bitrate, ABR) работает так: плеер измеряет фактическую скорость загрузки сегментов и оценку буфера, после чего выбирает наивысший вариант качества, который канал способен доставить без прерываний. При ухудшении сети плеер переключается на более низкий битрейт, при улучшении — на более высокий. Набор подготовленных вариантов качества называют «лестницей битрейтов» (bitrate ladder).

Существуют альтернативные технологии доставки. В таблице приведено их сравнение.

Таблица 2 — Сравнение технологий доставки видео

Технология	Транспорт	Адаптивный битрейт	Применимость
Progressive download	Один файл по HTTP	Нет	Простые случаи, малые видео
HLS	Сегменты + .m3u8 по HTTP	Да	Широкая (Apple, браузеры, hls.js)
MPEG-DASH	Сегменты + .mpd по HTTP	Да	Открытый стандарт, аналог HLS
RTMP	Постоянное TCP-соединение	Ограниченно	Устаревает, требует Flash/спец-сервер

Для данной работы выбран HLS как наиболее распространённый стандарт, поддерживаемый напрямую в Safari/iOS и через библиотеку hls.js — в остальных браузерах, что соответствует заданию варианта 33.

1.1.1 Подходы к защите контента

Поскольку сегменты раздаются по обычным HTTP-ссылкам, отдельной задачей становится защита контента — ограничение доступа к сегментам только для авторизованных пользователей. Сравнение распространённых подходов приведено в таблице.

Таблица 3 — Сравнение подходов к защите видео-контента

Подход	Принцип	Достоинства	Недостатки
Подписанные URL	Ссылка с подписью и сроком	Простота, без БД на раздаче	Ссылку можно переслать до истечения
Токен доступа	Токен в заголовке/cookie, проверка на раздаче	Гибкость, привязка к уроку/пользователю	Нужна общая логика проверки
AES-128 (HLS)	Шифрование сегментов, ключ по URL	Контент зашифрован	Защита ключа — отдельная задача
DRM	Widevine/FairPlay/PlayReady	Промышленная защита	Сложность, лицензии, инфраструктура

В работе применён подход на основе подписанного токена, передаваемого в заголовке Authorization: он обеспечивает привязку доступа к конкретному уроку и пользователю, ограничение по времени и единую схему проверки на стороне раздающего Go-сервиса. Это разумный баланс между надёжностью и сложностью реализации для учебного проекта; промышленные DRM и AES-128 обозначены как перспектива развития.

1.2 Анализ аналогов

Рассмотрены существующие образовательные платформы и видеосервисы с точки зрения доставки и защиты видео.

Таблица 4 — Сравнение аналогов по доставке и защите видео

Платформа	Доставка видео	Защита контента	Особенности
Coursera	HLS/DASH, адаптивный битрейт	Подписанные URL, аккаунт	Крупные курсы, сертификаты
Stepik	HLS	Привязка к сессии	Тесты, практика по коду
Udemy	HLS, адаптивный битрейт	Signed URL, watermark	Маркетплейс курсов
YouTube	DASH/HLS	Signed URL, токены	Универсальный видеохостинг

Общим для всех решений является применение адаптивного стриминга (HLS/DASH) и контроля доступа через подписанные ссылки или токены. Эти проверенные подходы и положены в основу разрабатываемой системы: HLS для доставки и подписанные токены для защиты.

1.3 Постановка задачи и требования

На основе анализа сформулированы требования к программному продукту.

1.3.1 Функциональные требования

- регистрация и аутентификация пользователей, разделение ролей: студент и преподаватель;
- управление курсами: создание курса, модулей и уроков преподавателем;

- загрузка преподавателем видео в урок и его автоматическая обработка в HLS с несколькими битрейтами;
- запись студента на курс и защищённый просмотр видео с адаптивным качеством;
- тестирование знаний: тесты с вопросами, подсчёт результата, фиксация попыток;
- учёт прогресса прохождения уроков.

1.3.2 Нефункциональные требования

- производительность раздачи видео (кэширование сегментов, эмуляция CDN);
- защита контента: сегменты недоступны без действительного токена;
- контейнеризация и развёртывание одной командой (Docker Compose);
- покрытие кода модульными тестами; статический анализ безопасности (SAST) и проверка зависимостей (SCA);
- документированность: README, документация API, диаграммы.

1.3.3 Действующие лица и сценарии использования

В системе выделены три роли: студент, преподаватель и администратор. Их основные сценарии приведены в таблице.

Таблица 5 — Роли и сценарии использования

Роль	Основные сценарии
Студент	Регистрация и вход; просмотр опубликованных курсов; запись на курс; просмотр видео-уроков; прохождение тестов; накопление прогресса
Преподаватель	Создание курсов, модулей, уроков; загрузка видео; создание тестов; удаление своего контента; предпросмотр уроков
Администратор	Управление всеми данными через административную панель Django (пользователи, курсы, тесты, записи)

Типовой сквозной сценарий выглядит так: преподаватель создаёт курс и наполняет его уроками, загружает видео (которое автоматически обрабатывается в адаптивный HLS); студент находит опубликованный курс, записывается на него, смотрит видео с качеством, подстраивающимся под канал, и проходит тесты; система фиксирует его прогресс. Этот сценарий и стал основой для интеграционной проверки (раздел 5.2).

2 ПРОЕКТНАЯ ЧАСТЬ

2.1 Общая архитектура

Система спроектирована как набор слабосвязанных сервисов, каждый из которых решает свою задачу. Такой микросервисный подход соответствует назначению языков из задания: Go применяется для сетевых сервисов (стриминг, транскодинг), Python (Django) — для бизнес-логики и данных, Nginx — для маршрутизации и кэширования.

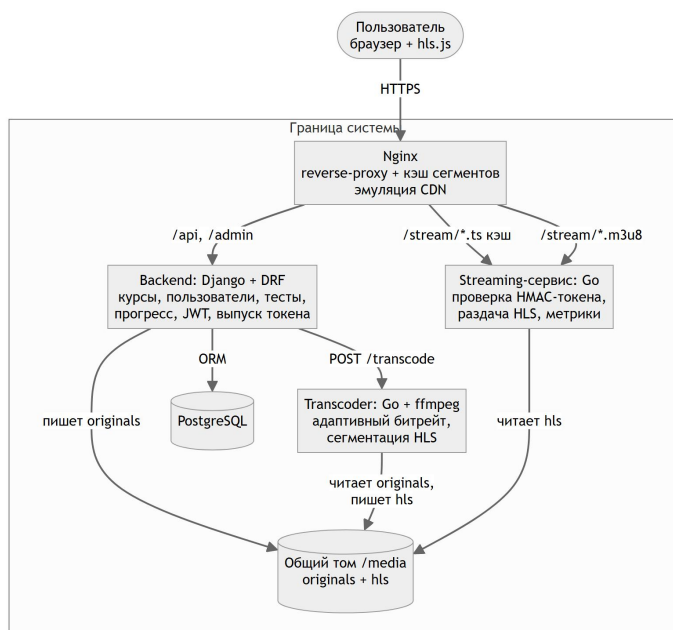


Рисунок 1 — Контейнерная диаграмма системы (нотация C4)

Компоненты системы:

- Nginx — единая точка входа: проксирует запросы к бэкенду и стриминг-сервису, отдаёт статику фронтенда, кэширует видео-сегменты (эмуляция CDN);
- Backend (Django + DRF) — REST API управления курсами, пользователями, тестами, прогрессом; аутентификация по JWT; выпуск подписанных токенов доступа к видео;
- Streaming-сервис (Go) — проверка токена и защищённая раздача HLS, экспорт метрик;

- Transcoder (Go + ffmpeg) — преобразование загруженного видео в адаптивный HLS;
- PostgreSQL — хранилище данных;
- общий том /media — обмен файлами (исходники и готовый HLS) между бэкендом и Go-сервисами.

Зоны ответственности и технологии компонентов сведены в таблице.

Таблица 6 — Зоны ответственности компонентов

Компонент	Технологии	Ответственность	Порт
Nginx	nginx	Прокси, статика, кэш сегментов (CDN)	80
Backend	Django, DRF, JWT	API, данные, токены доступа	8000
Streaming	Go (stdlib)	Проверка токена, раздача HLS, метрики	8081
Transcoder	Go, ffmpeg	Транскодинг в адаптивный HLS	8082
PostgreSQL	PostgreSQL 16	Хранение данных	5432

2.2 Модель данных

Доменная модель отражает структуру учебного контента и активность пользователей. Курс состоит из модулей, модуль — из уроков; к уроку привязаны видео и тесты. Студенты записываются на курсы, проходят тесты (фиксируются попытки) и накапливают прогресс по урокам.

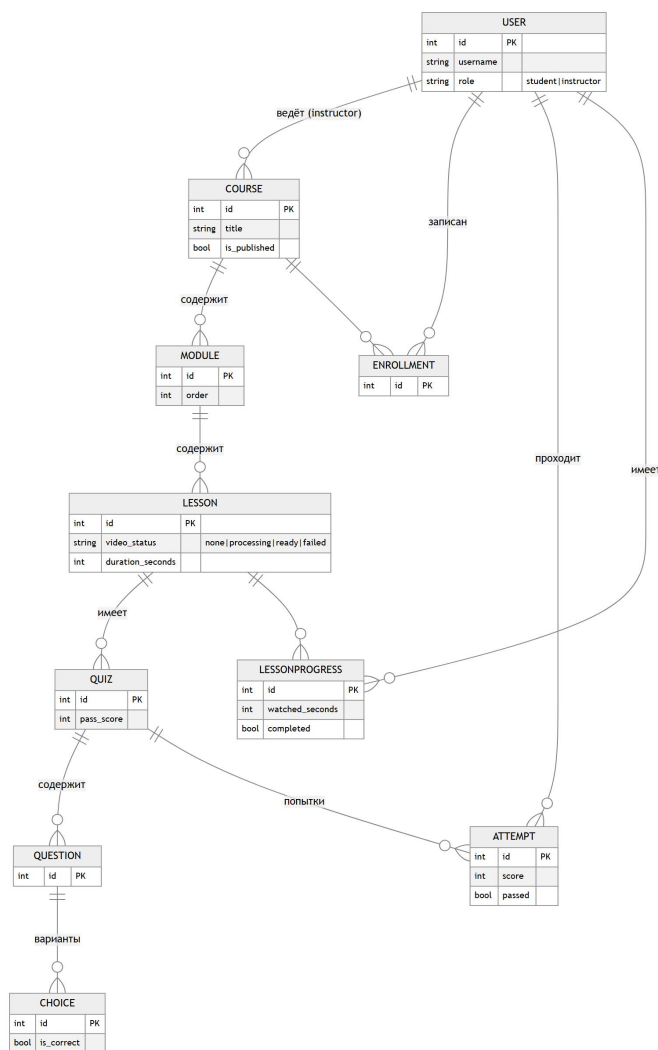


Рисунок 2 — Диаграмма «сущность-связь» модели данных

Роль пользователя (студент или преподаватель) определяет права: преподаватель управляет своими курсами и загружает видео, студент записывается на курсы и проходит обучение.

Состав ключевых сущностей приведён в таблицах. Сущность «Урок» хранит статус обработки видео, отражающий стадию транскодинга.

Таблица 7 — Сущность «Курс» (Course)

Поле	Тип	Назначение
title	строка	Название курса
description	текст	Описание
instructor	FK → User	Автор (преподаватель)
is_published	логический	Опубликован ли курс
created_at	дата/время	Дата создания

Таблица 8 — Сущность «Урок» (Lesson)

Поле	Тип	Назначение
module	FK → Module	Модуль, к которому относится урок
title	строка	Название урока
content	текст	Текстовое содержание
order	целое	Порядок в модуле
video_status	перечисление	none / processing / ready / failed
duration_seconds	целое	Длительность видео, с

Остальные сущности: «Модуль» (Module) группирует уроки внутри курса с указанием порядка; «Запись на курс» (Enrollment) связывает студента и курс с ограничением уникальности пары; «Тест» (Quiz), «Вопрос» (Question) и «Вариант» (Choice) описывают проверочные задания, причём признак правильности варианта скрывается от студента; «Попытка» (Attempt) фиксирует результат прохождения теста; «Прогресс урока» (LessonProgress) хранит просмотренное время и факт завершения для каждой пары студент—урок.

2.3 Проектирование защиты контента

Защита контента построена на симметричной схеме подписанных токенов. При запросе на просмотр Django проверяет, что пользователь записан на курс (или является его автором), и выпускает компактный токен, подписанный по алгоритму HMAC-SHA256 на общем секрете. Токен содержит идентификатор урока, идентификатор пользователя и время истечения.

Go-стриминг при каждом запросе плейлиста и сегмента повторяет ту же проверку: пересчитывает HMAC, сверяет подпись, контролирует срок действия и совпадение урока. Таким образом, прямая ссылка на сегмент без

действительного токена (или с истёкшим) получает ответ 403. Секрет известен только серверным компонентам и наружу не передаётся.

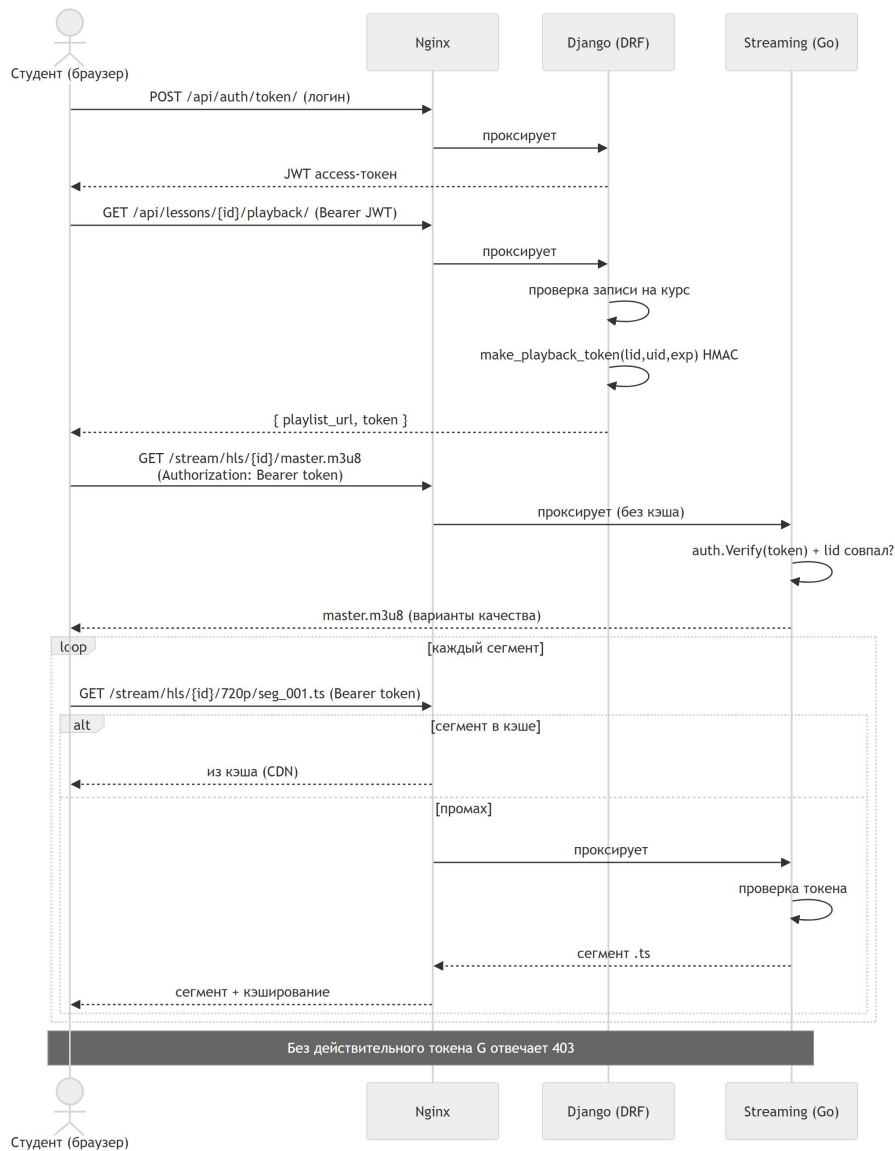


Рисунок 3 — Диаграмма последовательности: защищённый просмотр видео

Токен передаётся в заголовке Authorization при каждом запросе плеера, поэтому он распространяется и на главный плейлист, и на плейлисты вариантов качества, и на сегменты. Кэш Nginx учитывает токен в ключе кэширования, что исключает выдачу сегмента из кэша в обход проверки доступа.

2.4 Проектирование транскодинга

После загрузки видео преподавателем оно асинхронно преобразуется в адаптивный HLS. Backend сохраняет исходный файл в общий том и ставит

задачу транскодеру, который немедленно отвечает о приёме и выполняет обработку в фоне, не блокируя веб-процессы. По завершении транскодер вызывает callback бэкенда и тот переводит урок в статус «готово».

Транскодер с помощью ffmpeg формирует три варианта качества (360p, 720p, 1080p): для каждого создаются сегменты .ts и плейлист, а затем — общий master-плейлист со ссылками на варианты и указанием битрейта. Длительность исходного видео определяется через ffprobe.

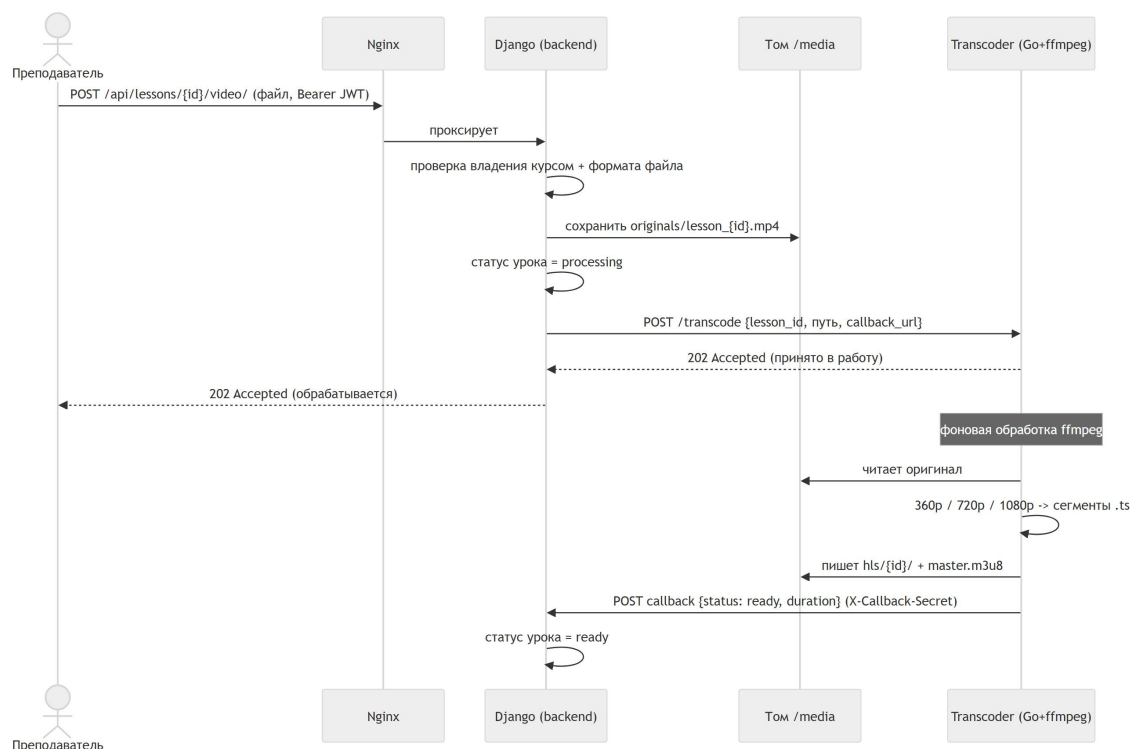


Рисунок 4 — Диаграмма последовательности: загрузка видео и асинхронный транскодинг

Асинхронная схема с обратным вызовом (callback) выбрана сознательно: транскодинг — длительная операция, и его синхронное выполнение в HTTP-запросе блокировало бы рабочий процесс веб-сервера и могло превысить его тайм-аут. Принимая задачу немедленно и отвечая статусом 202, система сохраняет отзывчивость, а статус урока обновляется по факту завершения.

2.5 Проектирование API

Прикладной интерфейс реализован в стиле REST на базе Django REST Framework. Аутентификация — по JWT (access/refresh-токены). Эндпоинты сгруппированы по доменам: пользователи, курсы, модули, уроки, тесты, прогресс, стриминг. Документация API формируется автоматически в формате OpenAPI и доступна в интерактивном виде (Swagger UI). Перечень основных групп эндпоинтов приведён в таблице.

Таблица 9 — Основные группы эндпоинтов REST API

Группа	Назначение
auth	Регистрация, вход (JWT), профиль
courses	CRUD курсов, запись на курс
modules / lessons	CRUD модулей и уроков
quizzes	Тесты, прохождение, подсчёт результата
progress	Прогресс прохождения уроков
streaming	Загрузка видео, выдача токена просмотра

Полный перечень основных эндпоинтов приведён в таблице.

Таблица 10 — Перечень основных эндпоинтов REST API

Метод и путь	Назначение
POST /api/auth/register/	Регистрация пользователя
POST /api/auth/token/	Получение JWT (вход)
POST /api/auth/token/refresh/	Обновление access-токена
GET /api/auth/me/	Профиль текущего пользователя
GET/POST /api/courses/	Список / создание курсов
GET/PUT/DELETE /api/courses/{id}/	Чтение / изменение / удаление курса
POST /api/courses/{id}/enroll/	Запись на курс
GET/POST /api/modules/	Список / создание модулей
GET/POST /api/lessons/	Список / создание уроков
GET/POST /api/quizzes/	Список / создание тестов

Метод и путь	Назначение
POST /api/quizzes/{id}/submit/	Прохождение теста
GET/POST /api/progress/	Чтение / обновление прогресса
POST /api/lessons/{id}/video/	Загрузка видео урока
GET /api/lessons/{id}/playback/	Выдача подписанной ссылки на просмотр
GET /api/docs/	Интерактивная документация (Swagger UI)

2.6 Развёртывание

Все компоненты упакованы в контейнеры и оркеструются через Docker Compose. Единственный публикуемый наружу порт — порт Nginx; остальные сервисы доступны только внутри сети Docker. Данные сохраняются в именованных томах: media (исходники и HLS) и pgdata (база данных).

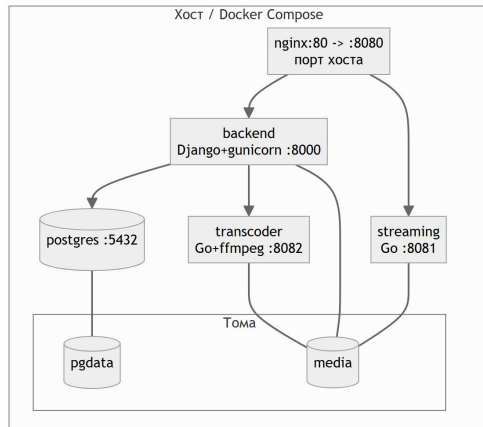


Рисунок 5 — Диаграмма развёртывания (Docker Compose)

3 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

Выбор технологий продиктован заданием варианта 33 (Go + Python + Nginx) и характером задач каждого компонента.

3.1 Go для стриминга и транскодинга

Go выбран для сетевых сервисов согласно методическим указаниям. Его преимущества здесь: лёгкие горутины для конкурентной обработки запросов, быстрый компилируемый код, богатая стандартная библиотека (net/http, crypto/hmac, os/exec). Сервисы реализованы исключительно на стандартной библиотеке, без внешних зависимостей, что упрощает сборку и снижает поверхность атаки.

Для задачи раздачи видео важна именно конкурентная модель Go: каждое соединение обслуживается отдельной горутинной — лёгким потоком, которых среда исполнения способна поддерживать десятками тысяч при умеренном потреблении памяти. Это позволяет одному экземпляру стриминг-сервиса обслуживать множество одновременных зрителей без сложной настройки пулов потоков. Компиляция в единый статический бинарный файл упрощает упаковку сервиса в минимальный контейнерный образ.

3.2 Django и DRF для бэкенда

Python с фреймворком Django выбран для бизнес-логики (по заданию — Django). Django предоставляет зрелую ORM, систему миграций, готовую административную панель и обширную экосистему. Django REST Framework добавляет удобные средства построения REST API, сериализацию и систему прав доступа. Аутентификация по JWT реализована библиотекой `djangorestframework-simplejwt`, документация — `drf-spectacular`.

В качестве альтернативы рассматривался FastAPI. Сравнение приведено в таблице; выбор сделан в пользу Django согласно заданию и ради готовой админ-панели и ORM.

Таблица 11 — Сравнение Django и FastAPI для бэкенда

Критерий	Django + DRF	FastAPI
ORM и миграции	Встроены (Django ORM)	Внешние (SQLAlchemy и др.)
Админ-панель	Встроена	Отсутствует
Документация API	drf-spectacular (OpenAPI)	Встроенная (OpenAPI)
Зрелость экосистемы	Очень высокая	Высокая, моложе
Соответствие заданию	Соответствует (Django)	Отклонение от текста

3.3 Nginx

Nginx выступает обратным прокси и точкой входа, а также кэширует видео-сегменты, эмулируя поведение CDN: повторные запросы популярных сегментов обслуживаются из кэша, снижая нагрузку на стриминг-сервис. Кэш настроен так, что в ключ входит токен доступа, поэтому кэширование не нарушает защиту контента.

Выбор Nginx обусловлен его эффективностью при отдаче статического и кэшированного контента, зрелостью и широким распространением. В реальной системе этот слой естественным образом заменяется или дополняется промышленным CDN: модель «обратный прокси с кэшем сегментов» полностью соответствует тому, как работают сети доставки контента, поэтому переход на внешний CDN не требует изменения логики приложения. Под путём /stream наружу публикуются только файлы HLS, что ограничивает поверхность атаки.

3.4 Вспомогательные технологии

- PostgreSQL — надёжная реляционная СУБД для хранения данных;
- ffmpeg — индустриальный инструмент транскодинга и сегментации видео;

- Docker и Docker Compose — контейнеризация и оркестрация развёртывания;
- hls.js — библиотека воспроизведения HLS в браузере на стороне фронтенда.

Основные используемые технологии и их версии приведены в таблице.

Таблица 12 — Используемые технологии и версии

Технология	Версия	Назначение
Python	3.12	Язык бэкенда
Django	5.1	Веб-фреймворк
Django REST Framework	3.15	REST API
djangorestframework-simplejwt	5.3	JWT-аутентификация
drf-spectacular	0.28	Документация OpenAPI
Go	1.26	Язык стриминг-сервисов
PostgreSQL	16	СУБД
Nginx	1.27	Обратный прокси, кэш
ffmpeg	(образ Alpine)	Транскодинг видео
hls.js	1.5	Воспроизведение HLS в браузере

3.5 Соответствие техническому заданию

Итоговый стек полностью соответствует варианту 33: Go обеспечивает стриминг с адаптивным битрейтом (HLS) и транскодинг, Django — управление курсами, пользователями и тестами, Nginx выполняет роль прокси и кэширующего слоя (эмуляция CDN), реализована защита контента.

4 РЕАЛИЗАЦИЯ

Ниже описаны ключевые компоненты системы с фрагментами кода. Полные листинги приведены в приложении Б.

4.1 Модель пользователя и права доступа

Пользователь расширяет стандартную модель Django добавлением роли. Свойство `is_instructor` определяет права на управление курсами.

Листинг 2 — модель пользователя с ролью (accounts/models.py)

```
class User(AbstractUser):
    class Role(models.TextChoices):
        STUDENT = "student", "Студент"
        INSTRUCTOR = "instructor", "Преподаватель"
    role = models.CharField(max_length=16,
                           choices=Role.choices,
                           default=Role.STUDENT)

    @property
    def is_instructor(self) -> bool:
        return self.role == self.Role.INSTRUCTOR or
self.is_staff
```

Права разграничены классами разрешений DRF: запись доступна только преподавателям, изменять объект может только преподаватель-владелец курса.

Листинг 3 — классы прав доступа (courses/permissions.py)

```
class IsInstructorOrReadOnly(permissions.BasePermission):
    def has_permission(self, request, view):
        if request.method in permissions.SAFE_METHODS:
            return True
        return bool(request.user and
request.user.is_instructor)

class
IsOwnerInstructorOrReadOnly(permissions.BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in permissions.SAFE_METHODS:
            return True
        course = getattr(obj, "course", obj)
        return course.instructor_id == request.user.id
```

Поскольку объектная проверка не вызывается при создании нового объекта (объекта ещё нет), для модулей и уроков добавлена явная проверка принадлежности целевого курса текущему пользователю — иначе через API можно было бы добавить контент в чужой курс.

4.2 Выпуск и проверка токена доступа

Django формирует токен: полезная нагрузка (идентификатор урока, пользователя, срок) кодируется в base64url и подписывается HMAC-SHA256.

Листинг 4 — выпуск токена на стороне Django (streaming/tokens.py)

```
def make_playback_token(lesson_id, user_id, signing_key,
    ttl):
    payload = {"lid": lesson_id, "uid": user_id,
               "exp": int(time.time()) + ttl}
    payload_b64 = _b64url(json.dumps(payload).encode())
    sig = hmac.new(signing_key.encode(),
    payload_b64.encode(),
                       hashlib.sha256).digest()
    return f"{payload_b64}.{_b64url(sig)}"
```

Go-сервис повторяет проверку: пересчитывает подпись, сравнивает её в постоянном времени (hmac.Equal), проверяет срок действия и совпадение урока из пути запроса.

Листинг 5 — проверка токена на стороне Go (internal/auth/auth.go)

```
func Verify(token, signingKey string) (Claims, error) {
    dot := strings.IndexByte(token, '.')
    payloadB64, sigB64 := token[:dot], token[dot+1:]
    expected := sign([]byte(payloadB64), signingKey)
    got, _ := base64.RawURLEncoding.DecodeString(sigB64)
    if !hmac.Equal(expected, got) { return claims,
ErrSignature }
    // ... разбор payload, проверка claims.Exp ...
}
```

При запросе на просмотр бэкенд проверяет право доступа (запись на курс или владение) и готовность видео, после чего выпускает токен и формирует адрес главного плейлиста.

Листинг 6 — Выдача подписанной ссылки (streaming/views.py)

```
is_owner = lesson.course.instructor_id == request.user.id
is_enrolled = Enrollment.objects.filter(
    user=request.user, course=lesson.course).exists()
if not (is_owner or is_enrolled):
    return Response(status=403)
token = make_playback_token(lesson.id, request.user.id,
                             SIGNING_KEY, TTL)
playlist_url = f"{PUBLIC_URL}/hls/{lesson.id}/master.m3u8"
return Response({"playlist_url": playlist_url, "token":
token})
```

4.3 Защищённая раздача HLS

Стриминг-сервис извлекает токен из заголовка Authorization (или query-параметра), проверяет его и соответствие урока, защищается от выхода за пределы каталога (path traversal) и только затем отдаёт файл, попутно учитывая метрики.

Листинг 7 — ядро защищённой раздачи (internal/hls/hls.go)

```
claims, err := auth.Verify(token, signingKey)
if err != nil || claims.LessonID != lessonID {
    deny(w, "access denied"); return
}
target := filepath.Join(base, filepath.Clean("/"+file))
if !strings.HasPrefix(target, base) {
    http.Error(w, "forbidden", 403); return
}
http.ServeFile(w, r, target)
```

4.4 Транскодинг в адаптивный HLS

Для каждого варианта качества вызывается ffmpeg; масштабирование выполняется без апскейла мелких исходников. Затем формируется master-плейлист.

*Листинг 8 — формирование варианта качества
(internal/transcode/transcode.go)*

```
args := []string{"-y", "-i", src,
    "-vf", fmt.Sprintf("scale=-2:min(ih,%d)", r.Height),
    "-c:v", "libx264", "-b:v", r.VBitrate,
    "-c:a", "aac", "-b:a", "128k",
    "-f", "hls", "-hls_time", "4",
    "-hls_playlist_type", "vod",
    "-hls_segment_filename", segPattern, indexPlaylist}
exec.Command("ffmpeg", args...).CombinedOutput()
```

После обработки всех вариантов формируется главный плейлист со ссылками на них и указанием битрейта и разрешения — именно его получает плеер.

*Листинг 9 — Формирование master-плейлиста
(internal/transcode/transcode.go)*

```
b.WriteString("#EXTM3U\n#EXT-X-VERSION:3\n")
for _, r := range rends {
    b.WriteString(fmt.Sprintf(
        "#EXT-X-STREAM-INF:BANDWIDTH=%d,RESOLUTION=%dx%d\n",
        r.Bandwidth, r.Width, r.Height))
    b.WriteString(fmt.Sprintf("%s/index.m3u8\n", r.Name))
}
os.WriteFile(masterPath, []byte(b.String()), 0o644)
```

Обработка асинхронна: транскодер принимает задачу, отвечает кодом 202 и запускает ffmpeg в фоне, а по завершении вызывает callback Django для смены статуса урока. Это исключает блокировку веб-процессов на время длительного транскодинга.

4.5 Nginx: проксирование и кэш-CDN

Под путём /stream наружу доступны только файлы HLS; служебные эндпоинты Go (метрики, healthcheck) не проксируются. Сегменты кэшируются, причём токен входит в ключ кэша.

Листинг 10 — кэширование сегментов с учётом токена (nginx.conf)

```
location ~* ^/stream/(hls/.+\.ts)$ {
    proxy_pass http://streaming/$1;
```

```

    proxy_cache hls_cache;
    proxy_cache_valid 200 10m;
    proxy_cache_key "$uri$is_args$args$http_authorization";
}

```

4.6 Фронтенд

Веб-клиент реализован без тяжёлых фреймворков. Воспроизведение HLS обеспечивает библиотека hls.js; токен доступа добавляется в заголовок каждого запроса плеера. Интерфейс ролевой: преподавателю доступны создание курсов, модулей и уроков, загрузка видео и удаление контента; студенту — запись на курс и просмотр.

Листинг 11 — подключение токена к плееру (web/app.js)

```

const hls = new Hls({ xhrSetup: (xhr) => {
    xhr.setRequestHeader("Authorization", "Bearer " +
token);
}});
hls.loadSource(playlistUrl);
hls.attachMedia(video);

```

Интерфейс определяет владельца курса сравнением автора курса с текущим пользователем и в зависимости от этого показывает либо элементы управления (создание, загрузка, удаление), либо кнопки записи и просмотра.

Листинг 12 — Определение владельца на фронтенде (web/app.js)

```

function isOwner(course) {
    return currentUser && course.instructor ===
currentUser.username;
}
// владельцу — форма загрузки и кнопки удаления;
// студенту — «Записаться» и «Смотреть» (если видео готово)

```

4.7 Контейнеризация и развёртывание

Все компоненты описаны в docker-compose.yml (база данных, бэкенд, транскодер, стриминг, Nginx). Запуск выполняется одной командой docker compose up. При старте бэкенд дожидается готовности БД, применяет миграции, собирает статику и наполняет демонстрационными данными. Go-

сервисы собираются мультистейдж-образом; в финальный образ включён ffmpeg.

Фрагмент описания сервисов в docker-compose.yml:

Листинг 13 — Фрагмент docker-compose.yml

```
services:
  db:      { image: postgres:16-alpine, ... }
  backend: { build: ./backend, depends_on: { db:
healthy } }
  transcoder:{ build: { context: ./go-services, args:
{ SERVICE: transcoder } } }
  streaming: { build: { context: ./go-services, args:
{ SERVICE: streaming } } }
  nginx:    { image: nginx:1.27-alpine, ports:
["${WEB_PORT:-8080}:80"] }
volumes: { pgdata: {}, media: {} }
```

Точка входа бэкенда дожидается БД, применяет миграции и запускает gunicorn с увеличенным тайм-аутом:

Листинг 14 — Точка входа бэкенда (entrypoint.sh)

```
while ! nc -z "$POSTGRES_HOST" "$POSTGRES_PORT"; do sleep
0.5; done
python manage.py migrate --noinput
python manage.py collectstatic --noinput || true
python manage.py seed_demo || true
exec gunicorn config.wsgi:application --bind 0.0.0.0:8000 \
--workers 3 --timeout 300
```

Непрерывная интеграция (GitHub Actions) запускает тесты и проверки безопасности на каждый push и pull request:

Листинг 15 — Этапы конвейера CI (.github/workflows/ci.yml)

```
jobs:
  backend-tests:    # pytest + coverage
  backend-security:# bandit (SAST) + pip-audit (SCA)
  go-tests:         # go vet + go test -race -cover
  go-security:      # govulncheck
```

4.8 Подсчёт результата теста

При прохождении теста сервер сверяет выбранные студентом варианты с правильными, вычисляет процент верных ответов и сравнивает его с порогом прохождения, после чего сохраняет попытку.

Листинг 16 — Подсчёт результата теста (quizzes/views.py)

```
correct = 0
for question in questions:
    chosen_id = answers.get(str(question.id))
    chosen = next((c for c in question.choices.all()
                    if c.id == chosen_id), None)
    if chosen and chosen.is_correct:
        correct += 1
score = round(correct / total * 100)
passed = score >= quiz.pass_score
Attempt.objects.create(quiz=quiz, user=request.user,
                        score=score, passed=passed)
```

4.9 Учёт прогресса

Прогресс по уроку обновляется идиомпотентно: повторная отправка по тому же уроку не создаёт дубликат, а обновляет существующую запись (просмотренное время, факт завершения).

Листинг 17 — Идиомпотентное обновление прогресса (progress/views.py)

```
obj, _ = LessonProgress.objects.update_or_create(
    user=request.user, lesson=data["lesson"],
    defaults={"watched_seconds": data.get("watched_seconds",
0),
              "completed": data.get("completed", False)})
```

Контроль версий ведётся в Git с использованием семантических коммитов и модели Git flow (основная ветка main и ветка разработки develop). История разработки приведена в приложении А.

5 ТЕСТИРОВАНИЕ

5.1 Модульное тестирование

Логика покрыта модульными тестами на двух стеках: `pytest` для бэкенда и `go test` для Go-сервисов. Тесты охватывают наиболее критичные части: схему токенов (с обеих сторон), права доступа API, подсчёт результатов тестов, защиту раздачи HLS.

Таблица 13 — Покрывание модульными тестами

Стек	Что проверяется	Тестов
<code>pytest</code>	Токены, права API, запись, подсчёт тестов, защита от чужих курсов	12
<code>go test (auth)</code>	Подпись, истечение, подмена, неверный ключ токена	5
<code>go test (hls)</code>	Раздача с токеном, отказ без токена, чужой урок, <code>path traversal</code>	5

Детальный перечень модульных тестов и проверяемых ими условий приведён в таблице.

Таблица 14 — Перечень модульных тестов

Тест	Что проверяет
<code>test_token_has_two_parts_and_payload</code>	Структура токена и корректность полезной нагрузки
<code>test_signature_matches_shared_secret</code>	Совпадение подписи с эталоном на общем секрете
<code>test_wrong_key_produces_different_signature</code>	Разные ключи дают разные подписи
<code>test_instructor_can_create_course</code>	Преподаватель может создать курс
<code>test_student_cannot_create_course</code>	Студенту запрещено создавать курс (403)
<code>test_student_sees_only_published_courses</code>	Студент видит только

Тест	Что проверяет
	опубликованные курсы
test_instructor_cannot_add_module_to_others_course	Запрет добавления модуля в чужой курс
test_instructor_cannot_add_lesson_to_others_module	Запрет добавления урока в чужой модуль
test_student_can_enroll	Студент записывается на курс
test_full_correct_passes / test_half_correct_score	Корректный подсчёт результата теста
test_choice_correctness_hidden_from_student	Правильность варианта скрыта от студента
TestVerifyValid / Expired / WrongKey / Malformed / Tampered	Go: валидный, истёкший, чужой, битый, подделанный токен
TestServesWithValidToken / DeniesWithoutToken	Go: раздача с токеном и отказ без него
TestDeniesTokenForOtherLesson / DeniesPathTraversal	Go: чужой урок и защита от обхода каталога

Все тесты проходят успешно. Результаты прогонов приведены на рисунках.

```
pytest — модульные тесты backend (Django)

===== test session starts =====
platform win32 -- Python 3.14.0, pytest-8.3.4, pluggy-1.6.0 -- C:\Users\User\Desktop\сама глазами\backend\.venv\Scripts\python.exe
cachedir: .pytest_cache
django: version: 5.1.4, settings: config.settings_test (from ini)
rootdir: C:\Users\User\Desktop\сама глазами\backend
configfile: pytest.ini
plugins: cov-6.0.0, django-4.9.0
collecting ... collected 12 items

tests/test_courses_api.py::test_instructor_can_create_course PASSED [ 8%]
tests/test_courses_api.py::test_student_cannot_create_course PASSED [ 16%]
tests/test_courses_api.py::test_student_sees_only_published_courses PASSED [ 25%]
tests/test_courses_api.py::test_instructor_cannot_add_module_to_others_course PASSED [ 33%]
tests/test_courses_api.py::test_instructor_cannot_add_lesson_to_others_module PASSED [ 41%]
tests/test_courses_api.py::test_student_can_enroll PASSED [ 50%]
tests/test_quizzes.py::test_full_correct_passes PASSED [ 58%]
tests/test_quizzes.py::test_half_correct_score PASSED [ 66%]
tests/test_quizzes.py::test_choice_correctness_hidden_from_student PASSED [ 75%]
tests/test_tokens.py::test_token_has_two_parts_and_payload PASSED [ 83%]
tests/test_tokens.py::test_signature_matches_shared_secret PASSED [ 91%]
tests/test_tokens.py::test_wrong_key_produces_different_signature PASSED [100%]

===== 12 passed, 15 warnings in 0.86s =====
```

Рисунок 6 — Результат прогона модульных тестов backend (pytest)

```
go test — модульные тесты Go-сервисов

?      videoplatform/cmd/streaming      [no test files]
?      videoplatform/cmd/transcoder     [no test files]
=== RUN   TestVerifyValid
--- PASS: TestVerifyValid (0.00s)
=== RUN   TestVerifyExpired
--- PASS: TestVerifyExpired (0.00s)
=== RUN   TestVerifyWrongKey
--- PASS: TestVerifyWrongKey (0.00s)
=== RUN   TestVerifyMalformed
--- PASS: TestVerifyMalformed (0.00s)
=== RUN   TestVerifyTampered
--- PASS: TestVerifyTampered (0.00s)
PASS
ok      videoplatform/internal/auth      0.361s
?      videoplatform/internal/config     [no test files]
=== RUN   TestServesWithValidToken
--- PASS: TestServesWithValidToken (0.00s)
=== RUN   TestDeniesWithoutToken
--- PASS: TestDeniesWithoutToken (0.00s)
=== RUN   TestDeniesTokenForOtherLesson
--- PASS: TestDeniesTokenForOtherLesson (0.00s)
=== RUN   TestDeniesPathTraversal
--- PASS: TestDeniesPathTraversal (0.00s)
PASS
ok      videoplatform/internal/hls        0.523s
?      videoplatform/internal/metrics    [no test files]
?      videoplatform/internal/transcode  [no test files]
```

Рисунок 7 — Результат прогона тестов Go-сервисов (go test)

5.2 Интеграционная проверка

Проведена сквозная проверка основного сценария на работающем стенде: преподаватель загружает видео — транскoder асинхронно формирует HLS — статус урока меняется на «готово» по callback — студент получает подписанную ссылку и смотрит видео с адаптивным качеством. Дополнительно подтверждено, что сегмент без действительного токена возвращает 403, в том числе при включённом кэше Nginx.

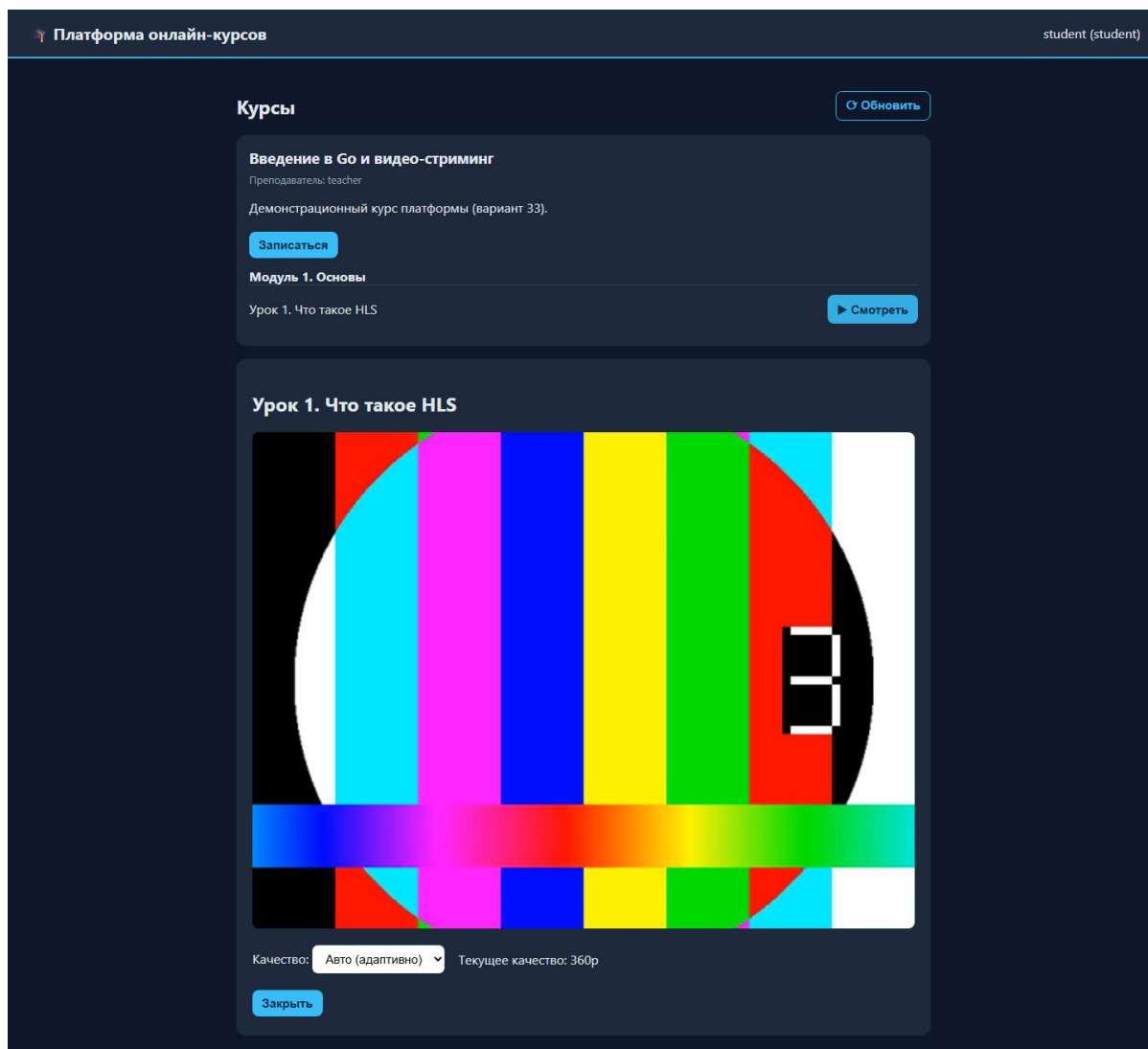


Рисунок 8 — Воспроизведение с адаптивным битрейтом (текущее качество 360p)

5.3 Безопасность

Безопасность обеспечивается на двух уровнях: автоматические проверки в конвейере CI и устранение дефектов, выявленных при разработке и код-ревью. Применяемые инструменты приведены в таблице.

Таблица 15 — Инструменты контроля безопасности

Инструмент	Тип	Язык	Что проверяет
bandit	SAST	Python	Небезопасные конструкции в коде
pip-audit	SCA	Python	Уязвимости в зависимостях PyPI
govulncheck	SCA	Go	Известные уязвимости в модулях Go
go vet	статан.	Go	Подозрительные конструкции

В ходе разработки и ревью выявлены и устранены реальные проблемы безопасности, в частности:

- кэш сегментов в Nginx изначально не учитывал токен в ключе — после первого запроса сегмент мог отдаваться без проверки доступа; устранено добавлением токена в ключ кэша;
- при создании модулей и уроков не срабатывала проверка владения — преподаватель мог добавить контент в чужой курс через API; устранено проверкой принадлежности;
- вход в административную панель за обратным прокси падал из-за отсутствия доверенных источников CSRF; устранено настройкой `CSRF_TRUSTED_ORIGINS`;
- запрещён запуск в производственном режиме с дефолтным секретным ключом.

5.4 Анализ производительности стриминга

Производительность доставки видео обеспечивается несколькими механизмами, сведёнными в таблице.

Таблица 16 — Механизмы обеспечения производительности

Механизм	Эффект
HLS-сегментация	Старт воспроизведения без загрузки всего файла; запрос только нужных фрагментов
Адаптивный битрейт	Автоснижение качества при слабом канале — без прерываний
Кэш сегментов в Nginx	Повторные запросы обслуживаются из кэша (эмуляция CDN), снижая нагрузку
Горутины в Go	Конкурентная обработка соединений, масштабируемость по числу зрителей
Асинхронный транскодинг	Веб-процессы не блокируются на время обработки видео

Таким образом, архитектура обеспечивает как отзывчивость при просмотре (быстрый старт, плавное переключение качества), так и масштабируемость при росте числа одновременных зрителей. Точные количественные показатели зависят от аппаратной конфигурации и выходят за рамки учебного проекта; для их измерения в перспективе применимы инструменты нагрузочного тестирования (например, vegeta или hey).

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы спроектирована и реализована платформа для онлайн-курсов с видео-стримингом (вариант 33). Поставленные задачи выполнены полностью:

- проведён анализ предметной области и аналогов, сформулированы требования;
- спроектирована микросервисная архитектура (Go + Django + Nginx), подготовлены диаграммы (C4, ER, последовательности);
- реализован полный продукт: адаптивный HLS-стриминг с защитой контента, асинхронный транскодинг на ffmpeg, REST API управления курсами, тестами и прогрессом, ролевой веб-интерфейс;
- обеспечено качество: модульные тесты (pytest и go test), статический анализ и проверка зависимостей в CI, выявлены и устранены реальные дефекты безопасности;
- подготовлена документация: README, автоматическая документация API (OpenAPI/Swagger), диаграммы, настоящая пояснительная записка; развёртывание выполняется одной командой через Docker Compose.

Полученный продукт демонстрирует владение современными языками (Go, Python), инструментами и методологиями разработки, а также прохождение полного цикла создания программного продукта от анализа до развёртывания.

Перспективы развития: интеграция с реальным CDN и DRM для усиленной защиты, поддержка прямых трансляций (live HLS), масштабирование транскодинга через очередь задач и пул воркеров, расширение аналитики обучения.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Макконнелл С. Совершенный код. — Microsoft Press, 2023.
2. Фаулер М. Архитектура корпоративных программных приложений. — Addison-Wesley, 2022.
3. RFC 8216. HTTP Live Streaming. — IETF, 2017. URL: <https://datatracker.ietf.org/doc/html/rfc8216>.
4. Документация Python 3.12. URL: <https://docs.python.org/3/>.
5. Документация Go. URL: <https://go.dev/doc/>.
6. Документация Django. URL: <https://docs.djangoproject.com/>.
7. Документация Django REST Framework. URL: <https://www.django-rest-framework.org/>.
8. Документация ffmpeg. URL: <https://ffmpeg.org/documentation.html>.
9. Документация Nginx. URL: <https://nginx.org/ru/docs/>.
10. Docker Documentation. URL: <https://docs.docker.com/>.
11. GitHub Actions Documentation. URL: <https://docs.github.com/actions>.
12. Refactoring Guru — паттерны проектирования. URL: <https://refactoring.guru/ru>.

ПРИЛОЖЕНИЕ А. Структура репозитория и история разработки

Проект организован как самодостаточный репозиторий, разворачиваемый через Docker Compose. Структура каталогов:

backend/	– Django + DRF (курсы, пользователи, тесты, токены)
go-services/	– Go-модуль: streaming и transcoder + общие пакеты
nginx/	– конфигурация обратного прокси и кэша
web/	– фронтенд-плеер на hls.js, ролевой интерфейс
docs/	– документация, диаграммы, скриншоты
.github/	– CI: тесты, SAST, SCA
docker-compose.yml	– оркестрация всех сервисов

Разработка велась в Git по модели Git flow с семантическими коммитами (feat, fix, test, ci, docs, chore), что обеспечивает прослеживаемость изменений и атомарность каждого логического шага.

ПРИЛОЖЕНИЕ Б. Документация API

Документация REST API формируется автоматически в формате OpenAPI и доступна в интерактивном виде через Swagger UI по адресу `/api/docs/`.

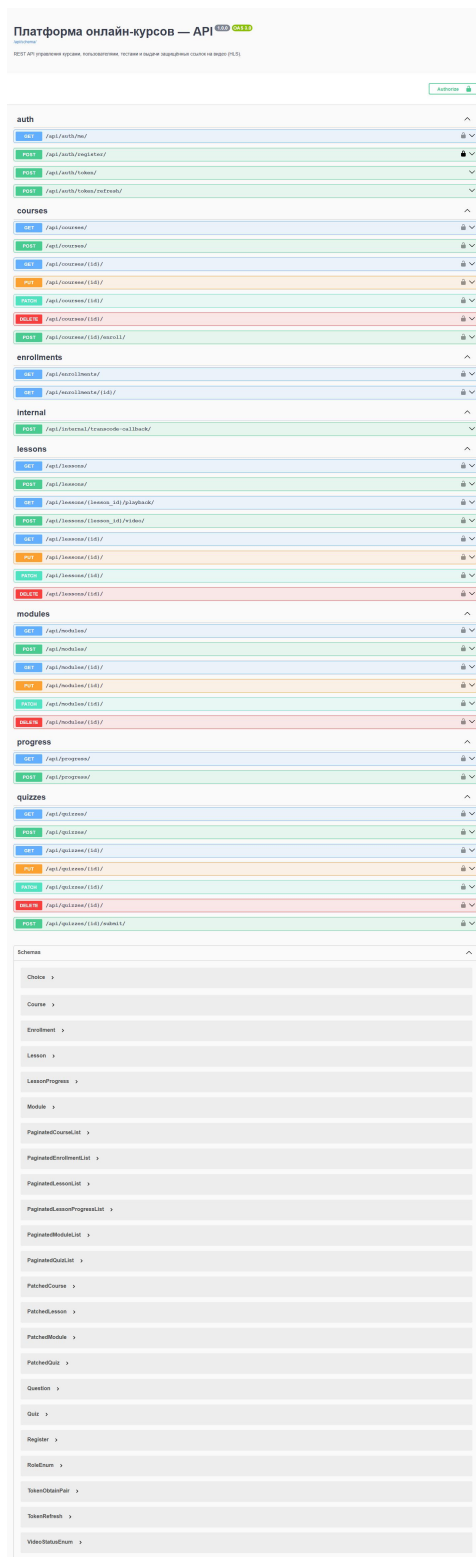


Рисунок 9 — Интерактивная документация API (Swagger UI)

ПРИЛОЖЕНИЕ В. Экранные формы

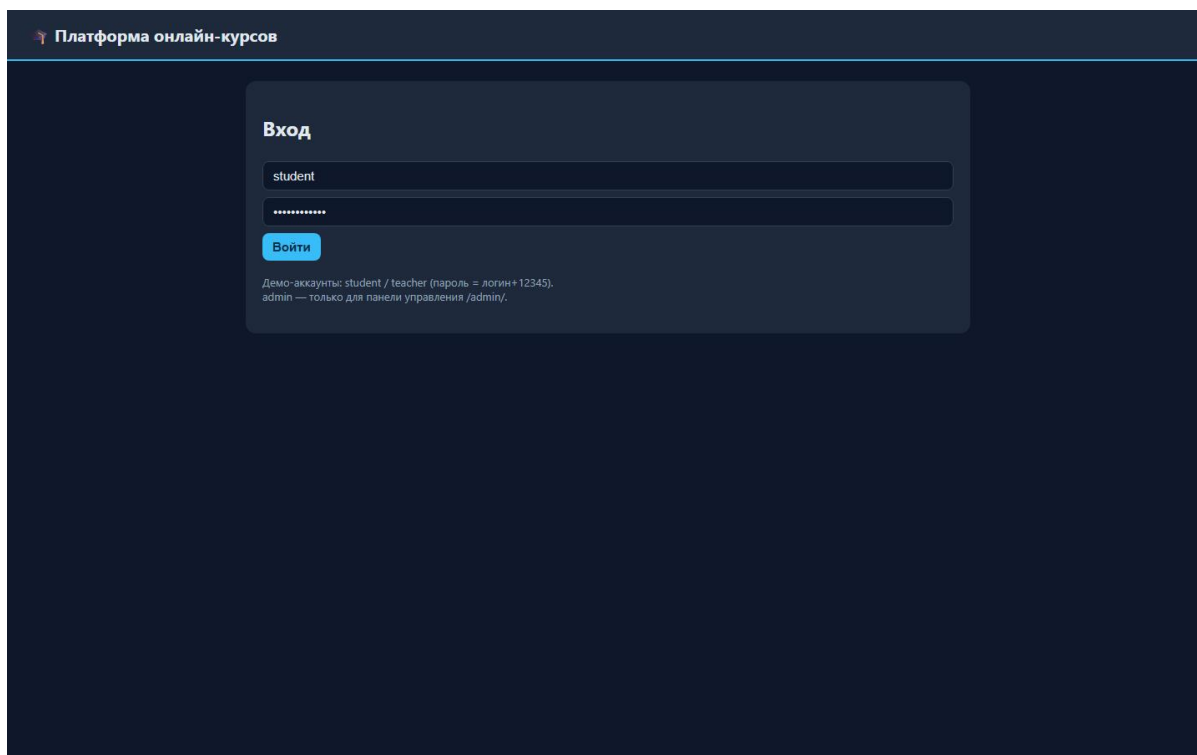


Рисунок 10 — Страница входа

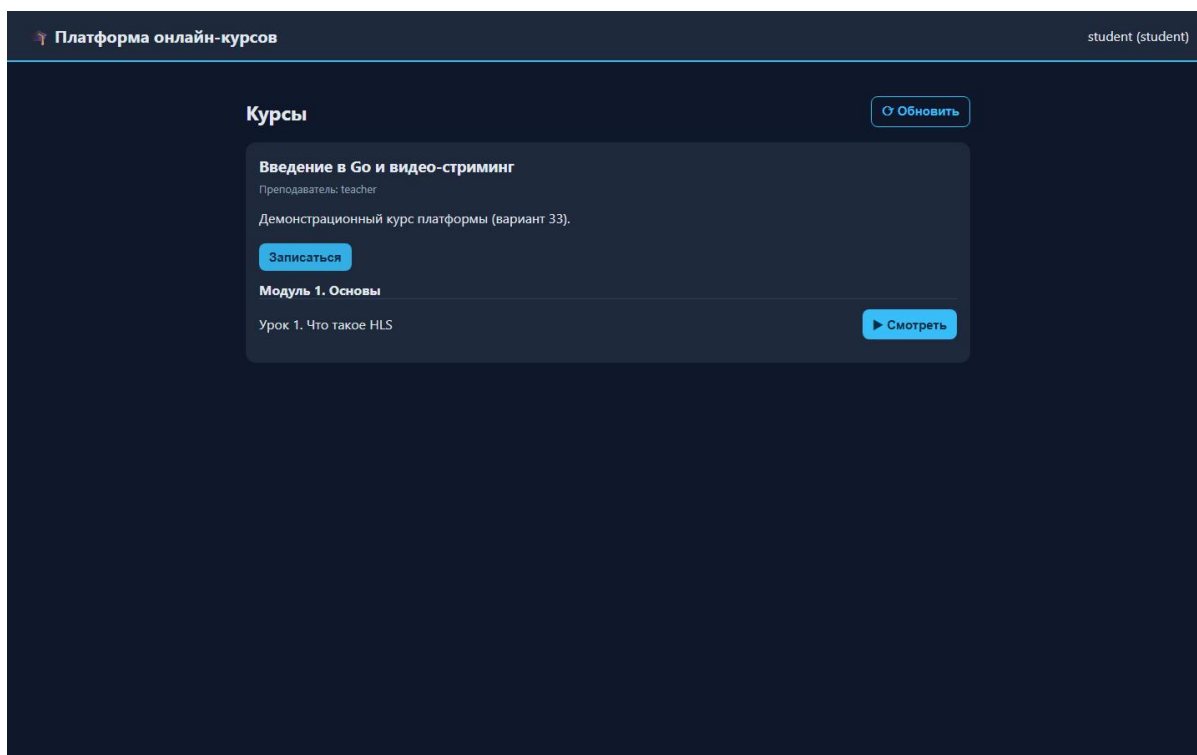


Рисунок 11 — Список курсов (роль «студент»)

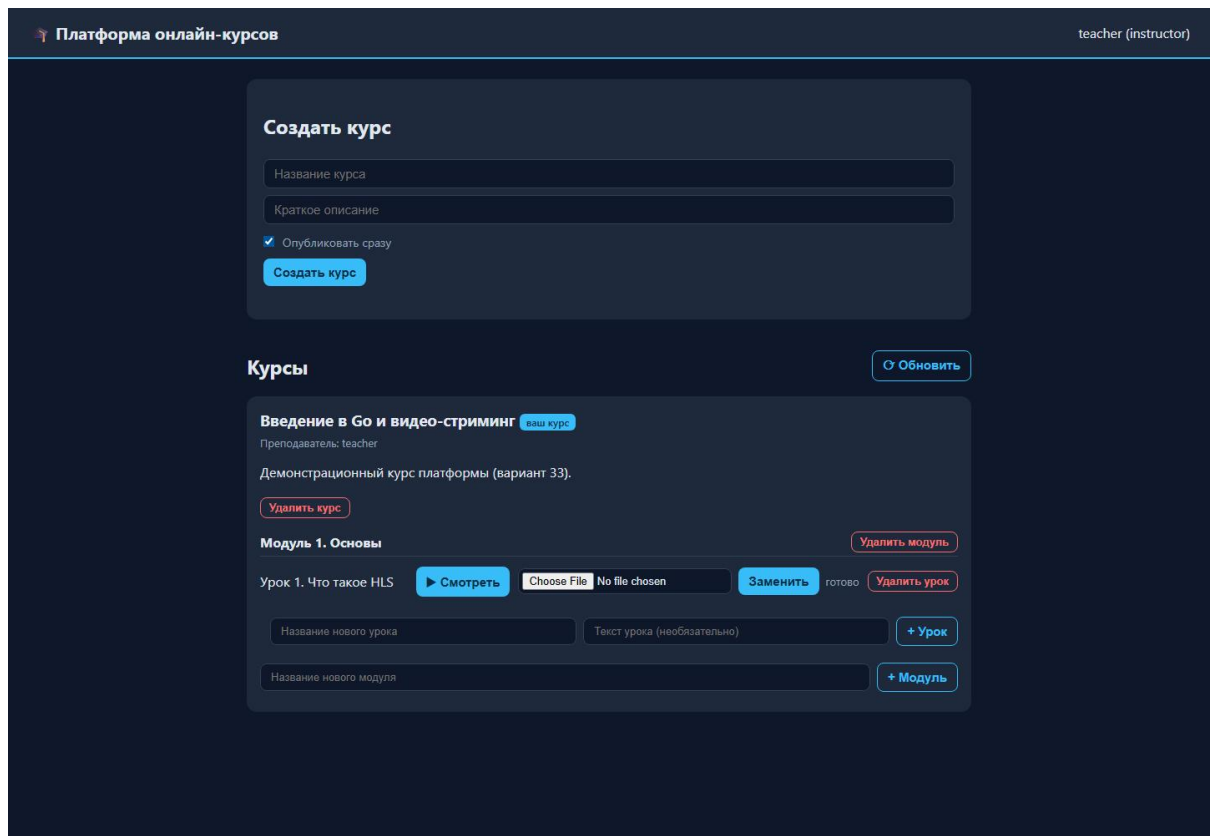


Рисунок 12 — Панель преподавателя: создание и управление контентом

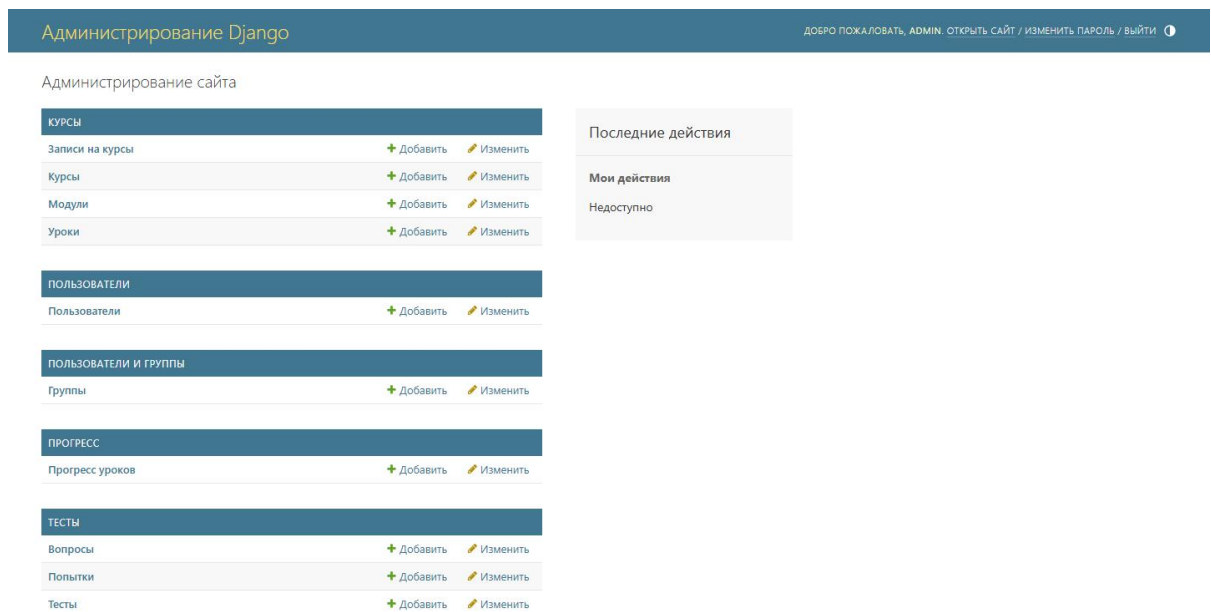


Рисунок 13 — Административная панель Django

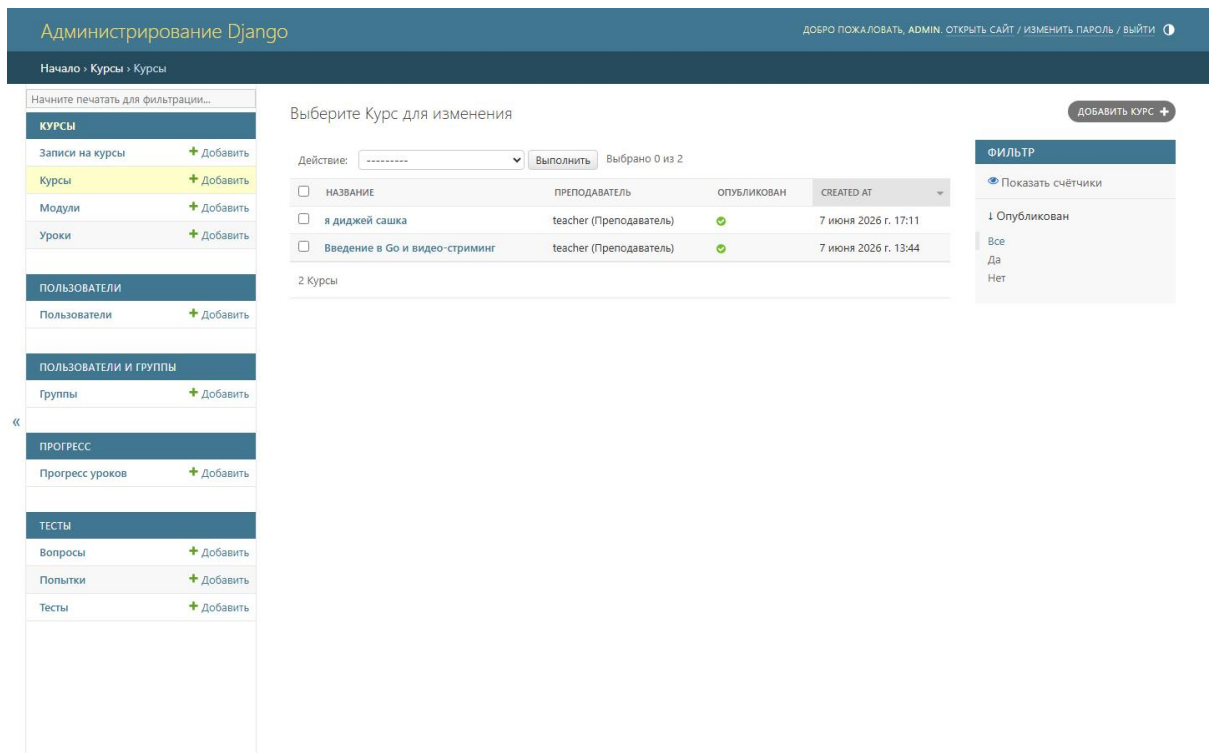


Рисунок 14 — Управление курсами в административной панели

ПРИЛОЖЕНИЕ Г. Руководство по запуску и проверке

Система разворачивается одной командой при наличии установленного Docker. Порядок запуска:

1. скопировать файл переменных окружения: `cp .env.example .env` (при необходимости изменить порт `WEB_PORT` и секреты);
2. собрать образы и запустить все сервисы: `docker compose up --build`;
3. дождаться готовности (в логах `backend` — запуск `gunicorn`);
4. открыть в браузере `http://localhost:8080` (или указанный `WEB_PORT`).

Демонстрационные учётные записи создаются автоматически: `student` / `teacher` (пароль — логин с суффиксом `12345`) и суперпользователь `admin` для административной панели `/admin/`. Полезные адреса:

Листинг 18 — Точки доступа к системе

```
http://localhost:8080          — веб-интерфейс (плеер, курсы)
http://localhost:8080/api/docs/ — документация API (Swagger UI)
http://localhost:8080/admin/   — административная панель Django
```

Запуск тестов выполняется командами `pytest` (в каталоге `backend`) и `go test ./...` (в каталоге `go-services`). Проверки безопасности (`bandit`, `pip-audit`, `govulncheck`) выполняются автоматически в конвейере непрерывной интеграции при отправке изменений в репозиторий.